MCP For Unity ドキュメント

目次

- 一、はじめに
- 二、使用方法
  - 2.1 エディタの使用
  - 2.2 ランタイムの使用
- 三、拡張性について
  - 3.1 エディタの拡張
  - 3.2 ランタイムの拡張
- 四、付録
- 五、AIアシスト開発

一、はじめに

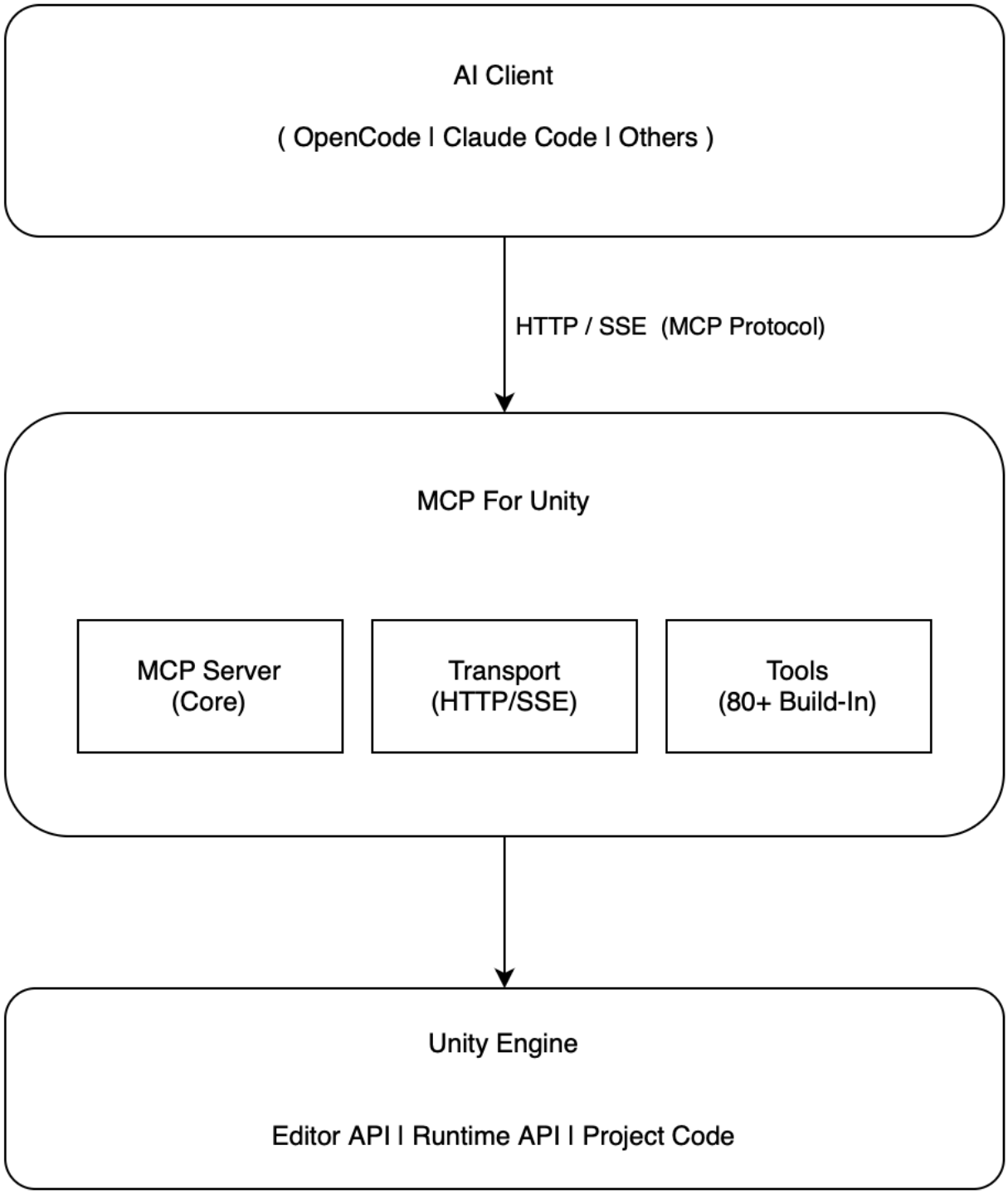
1.1 MCP For Unity とは

MCP For Unity は **Model Context Protocol (MCP)** に基づいた Unity 実装フレームワークで、Unity エディタとランタイムアプリケーションを AI エコシステムにシームレスに統合することを目的としています。

主な利点:

特徴	説明
純粋な C# 実装	外部ブリッジ不要、Python や他の言語依存なし
完全なツールチェーンアクセス	AI が Unity Engine API とプロジェクトコードを直接呼び出し可能
双方向通信	AI と Unity 間のリアルタイム対話をサポート
標準化されたプロトコル	Claude Desktop、VS Code MCP などの主要な AI クライアントと互換性あり

1.2 アーキテクチャ概要


AI Client
(OpenCode I Claude Code I Others)
HTTP / SSE (MCP Protocol)
MCP For Unity
MCP Server
(Core)
Transport
(HTTP/SSE)
Tools
(80+ Build-In)
Unity Engine
Editor API I Runtime API I Project Code

1.3 機能サポート

機能	サポート状況
Tools（ツール）	✅ 完全サポート
Resources（リソース）	✅ 完全サポート
Tasks（タスク）	✅ 完全サポート
Prompts（プロンプト）	❌ 非サポート

> ⚠️ **注意**: 現在のバージョンでは Prompts 機能はサポートされていません。

1.4 技術仕様

項目	説明
Unity バージョン	2021.3+
.NET バージョン	.NET Standard 2.1
転送プロトコル	HTTP + Server-Sent Events (SSE)
シリアライゼーション	Newtonsoft.Json
依存関係	`com.unity.nuget.newtonsoft-json`

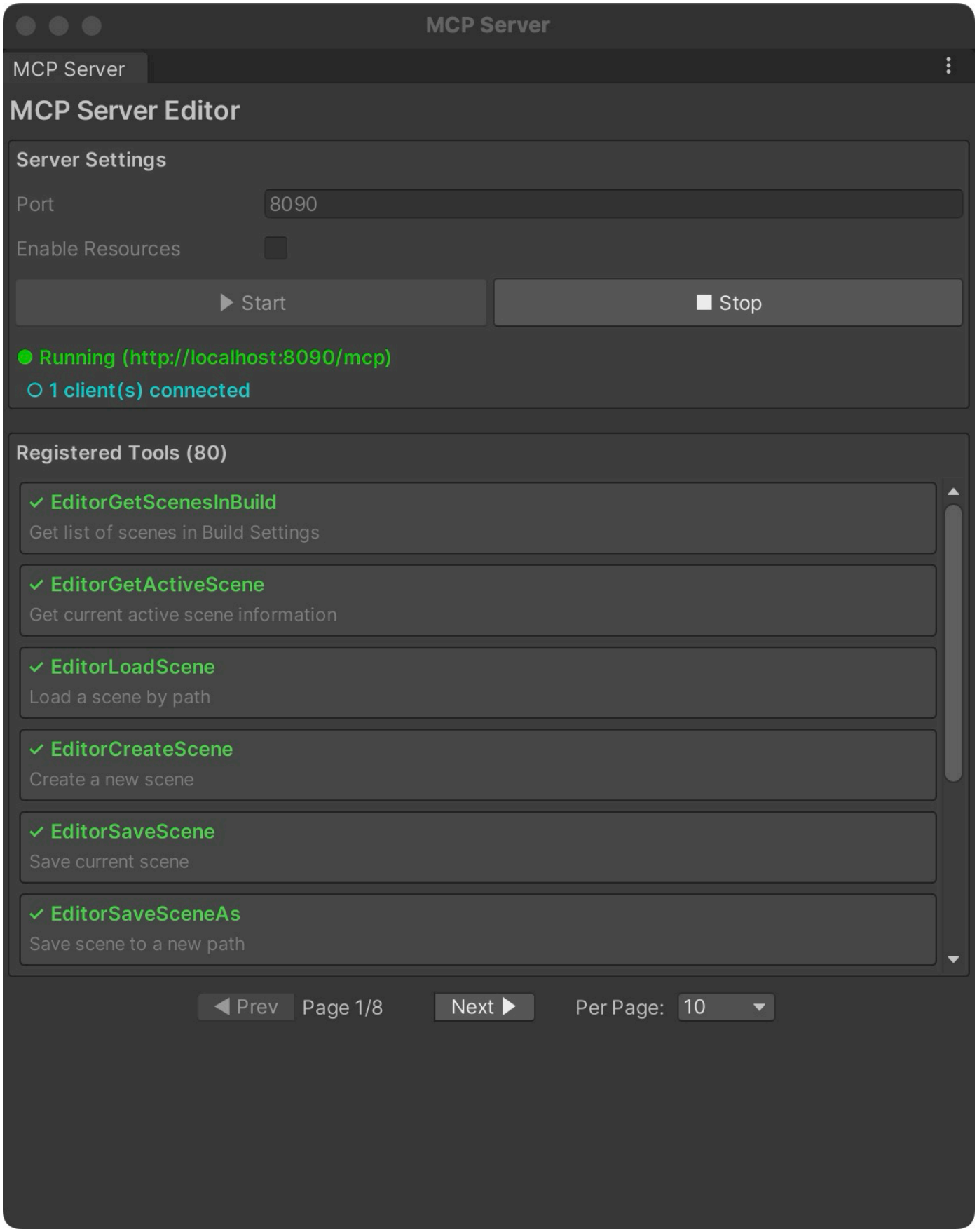
二、使用方法

2.1 エディタの使用

2.1.1 エディタサーバーの起動

方法1：エディタウィンドウから

1. メニューを開く：`Tools > MCP For Unity > Server Window`
2. サーバーパラメータを設定
3. **Start** ボタンをクリックして起動


MCP Server
MCP Server
MCP Server Editor
Server Settings
Port
8090
Enable Resources
Start
Stop
Running (http://localhost:8090/mcp)
1 client(s) connected
Registered Tools (80)
EditorGetScenesInBuild
Get list of scenes in Build Settings
EditorGetActiveScene
Get current active scene information
EditorLoadScene
Load a scene by path
EditorCreateScene
Create a new scene
EditorSaveScene
Save current scene
EditorSaveSceneAs
Save scene to a new path
Prev
Page 1/8
Next
Per Page:
10

方法2：コードから

```
using ModelContextProtocol.Editor;

// 起動（デフォルト設定を使用）
GlobalEditorMcpServer.StartServer();

// 停止
GlobalEditorMcpServer.StopServer();
```

2.1.2 設定オプション

エディタウィンドウで以下のオプションを設定できます:

オプション	デフォルト値	説明
Port	8090	サーバーポート
Enable Resources	false	リソースサービスを有効化
Enable File Watching	false	ファイル監視を有効化

2.1.3 組み込みツール一覧

MCP For Unity は **80 のエディタツール** を提供し、一般的な開発シナリオをカバーしています:

シーン管理 (10 ツール)

ツール名	説明
`EditorGetScenesInBuild`	Build Settings のシーンリストを取得
`EditorGetActiveScene`	現在のアクティブシーン情報を取得
`EditorLoadScene`	シーンをロード
`EditorCreateScene`	新しいシーンを作成
`EditorSaveScene`	現在のシーンを保存
`EditorSaveSceneAs`	シーンを別名で保存
`EditorCloseScene`	シーンを閉じる
`EditorAddSceneToBuild`	シーンを Build に追加
`EditorRemoveSceneFromBuild`	シーンを Build から削除
`EditorGetSceneHierarchy`	シーン階層構造を取得

GameObject 操作 (15 ツール)

ツール名	説明
`EditorCreateGameObject`	空の GameObject を作成
`EditorCreatePrimitive`	プリミティブ形状を作成
`EditorDuplicateGameObject`	GameObject を複製
`EditorDeleteGameObject`	GameObject を削除
`EditorFindGameObject`	GameObject を検索
`EditorFindGameObjectsByName`	名前で検索
`EditorFindGameObjectsByTag`	タグで検索
`EditorFindGameObjectsByLayer`	レイヤーで検索
`EditorGetGameObjectInfo`	詳細情報を取得
`EditorGetChildren`	子オブジェクトリストを取得
`EditorSetParent`	親を設定
`EditorSetTag`	タグを設定
`EditorSetLayer`	レイヤーを設定
`EditorSetStatic`	スタティックフラグを設定
`EditorSetActive`	アクティブ状態を設定

Transform 操作 (10 ツール)

ツール名	説明
`EditorSetPosition`	ワールド位置を設定
`EditorSetRotation`	ワールド回転を設定
`EditorSetScale`	スケールを設定
`EditorSetLocalPosition`	ローカル位置を設定
`EditorSetLocalRotation`	ローカル回転を設定
`EditorSetLocalScale`	ローカルスケールを設定
`EditorSetPositionAndRotation`	位置と回転を同時設定
`EditorTranslate`	相対移動
`EditorRotate`	相対回転
`EditorResetTransform`	トランスフォームをリセット

コンポーネント操作 (10 ツール)

ツール名	説明
`EditorAddComponent`	コンポーネントを追加
`EditorRemoveComponent`	コンポーネントを削除
`EditorGetComponent`	コンポーネント情報を取得
`EditorGetComponents`	すべてのコンポーネントを取得
`EditorGetComponentsInChildren`	子オブジェクトのコンポーネントを取得
`EditorSetComponentProperty`	コンポーネントプロパティを設定
`EditorGetComponentProperty`	コンポーネントプロパティを取得
`EditorHasComponent`	コンポーネントの有無を確認
`EditorGetRequiredComponent`	コンポーネントを取得または追加
`EditorSendComponentMessage`	メッセージを送信

コンパイル＆ビルド (10 ツール)

ツール名	説明
`EditorGetCompilationStatus`	コンパイルステータスを取得
`EditorIsCompiling`	コンパイル中かどうか
`EditorGetCompileErrors`	コンパイルエラーを取得
`EditorGetCompileWarnings`	コンパイル警告を取得
`EditorGetBuildTarget`	ビルドターゲットを取得
`EditorSetBuildTarget`	ビルドターゲットを設定
`EditorGetScriptingBackend`	スクリプティングバックエンドを取得
`EditorGetAssemblyDefinitions`	asmdef リストを取得
`EditorGetDefineSymbols`	定義シンボルを取得
`EditorSetDefineSymbols`	定義シンボルを設定

アセット管理 (15 ツール)

ツール名	説明
`EditorFindAssets`	アセットを検索
`EditorGetAssetInfo`	アセット詳細を取得
`EditorGetAssetPath`	アセットパスを取得
`EditorGetAssetDependencies`	アセット依存関係を取得
`EditorGetFolderContents`	フォルダ内容を取得
`EditorCreateFolder`	フォルダを作成
`EditorDeleteAsset`	アセットを削除
`EditorRenameAsset`	アセットをリネーム
`EditorMoveAsset`	アセットを移動
`EditorCopyAsset`	アセットをコピー
`EditorRefreshAssets`	アセットデータベースを更新
`EditorReimportAsset`	アセットを再インポート
`EditorGetAssetImportSettings`	インポート設定を取得
`EditorLoadAssetAtPath`	パスからアセットをロード
`EditorGetAllAssetPaths`	すべてのアセットパスを取得

プレハブ操作 (10 ツール)

ツール名	説明
`EditorInstantiatePrefab`	プレハブをインスタンス化
`EditorCreatePrefab`	プレハブを作成
`EditorApplyPrefab`	プレハブ変更を適用
`EditorRevertPrefab`	プレハブを元に戻す
`EditorUnpackPrefab`	プレハブをアンパック
`EditorGetPrefabInfo`	プレハブ情報を取得
`EditorGetPrefabType`	プレハブタイプを取得
`EditorIsPrefabInstance`	プレハブインスタンスかどうか
`EditorGetPrefabAssetPath`	プレハブアセットパスを取得
`EditorGetAllPrefabs`	すべてのプレハブを取得

2.1.4 実装場所

```
Assets/McpForUnity/Editor/
├── Tools/
│   └── EditorToolsList.cs       # 80 エディタツールの実装
├── GlobalEditorMcpServer.cs     # グローバルサーバー管理
└── Window/
    └── McpServerEditorWindow.cs # エディタウィンドウ
```

2.1.5 AI クライアントの接続

Claude Desktop 設定

Claude Desktop 設定ファイルを編集：

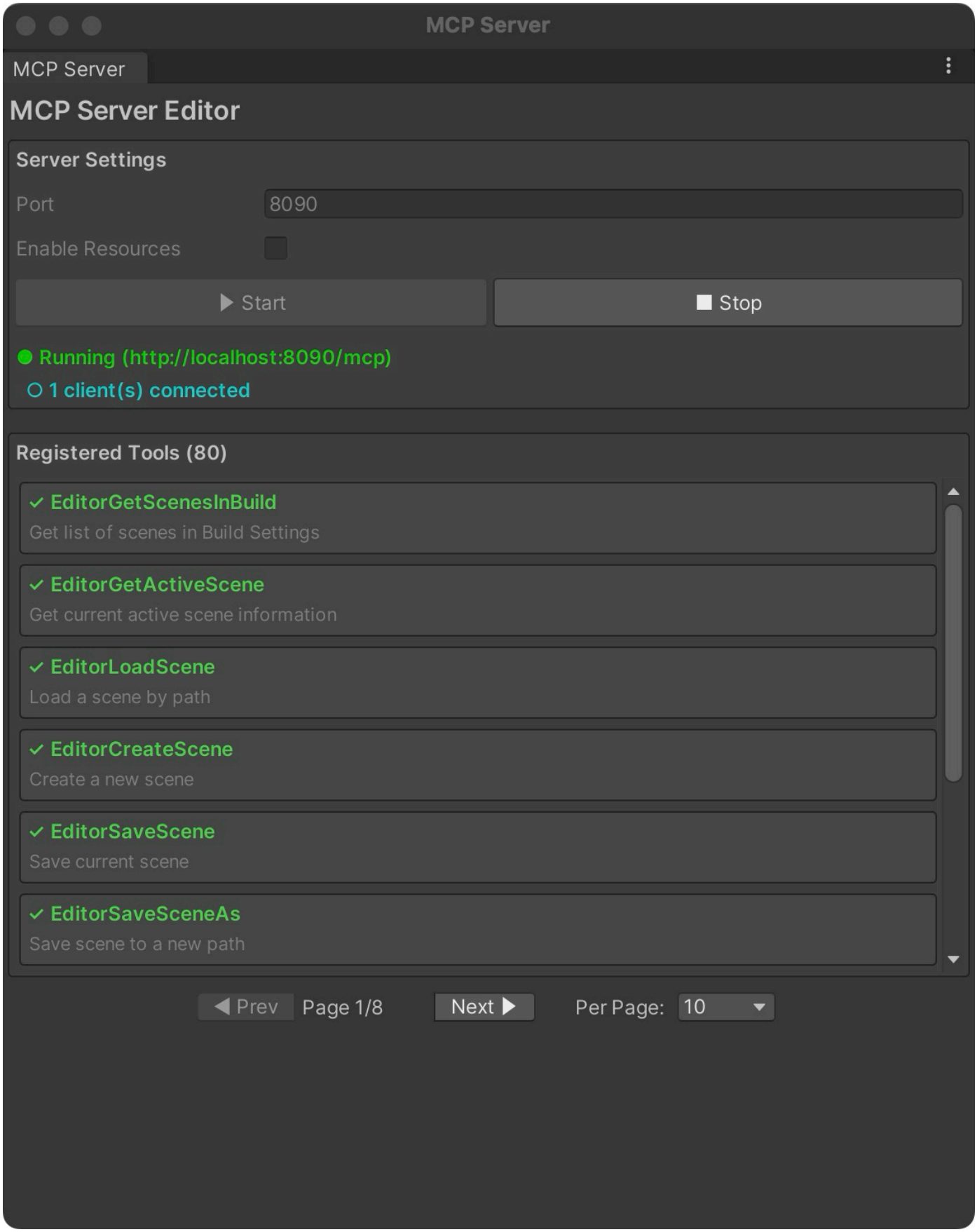
- macOS: `~/Library/Application Support/Claude/claude_desktop_config.json`
- Windows: `%APPDATA%\Claude\claude_desktop_config.json`

```json
{
  "mcpServers": {
    "unity-editor": {
      "url": "http://localhost:8090/mcp",
      "transport": "sse"
    }
  }
}
```

OpenCode 設定

プロジェクトルートに `.opencode/opencode.jsonc` ファイルを作成：

```json
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "UnityMCPServer": {
      "type": "remote",
      "url": "http://localhost:8090/mcp",
      "enabled": true
    }
  }
}
```



2.2 ランタイムの使用

2.2.1 ランタイムサーバーの起動

```
using System.Threading.Tasks;
```

```
using ModelContextProtocol.Unity;
using UnityEngine;

public class MCPManager : MonoBehaviour
{
    private McpServerHost _host;

    async void Start()
    {
        // 設定を作成
        var options = new McpServerHostOptions
        {
            Port = 3000,
            ServerName = "UnityMCP",
            ServerVersion = "1.0.0"
        };

        // サーバーを作成して起動
        _host = new McpServerHost(options);
        await _host.StartAsync();

        // カスタムツールを登録
        _host.Server.RegisterToolsFromClass(typeof(MyGameTools));

        Debug.Log($"MCP Server started at http://localhost:{options.Port}/mcp");
    }

    async void OnDestroy()
    {
        if (_host != null)
        {
            await _host.DisposeAsync();
        }
    }
}
```

> ⚠️ **重要**: ランタイムサーバーには**組み込みツールは含まれていません**。すべてのツールは `RegisterToolsFromClass()` または `AddCustomTool()` を使用して手動で登録する必要があります。

2.2.2 設定オプション

オプション	型	デフォルト値	説明
`Port`	int	3000	サーバーポート
`ServerName`	string	"UnityMCP"	サーバー名
`ServerVersion`	string	"1.0.0"	サーバーバージョン
`Instructions`	string	-	サーバー説明
`LogLevel`	LogLevel	Information	ログレベル

2.2.3 イベント購読

```
_host.OnServerStarted += () => Debug.Log("Server started");
_host.OnServerStopped += () => Debug.Log("Server stopped");
_host.OnServerError += (error) => Debug.LogError($"Server error: {error}");
```

2.2.4 カスタムツールの登録

```
// 方法1：クラス経由で登録
_host.Server.RegisterToolsFromClass(typeof(MyGameTools));

// 方法2：Lambda を直接追加
_host.AddCustomTool("my_tool", "ツールの説明", async (args, ct) =>
{
    string param = args?["param"]?.ToString();
    // 処理ロジック
    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = "結果" }
        }
    };
});
```

三、拡張性について

3.1 エディタの拡張

3.1.1 カスタムツールの定義

属性アノテーションを使用してツールを定義：

```
using ModelContextProtocol.Protocol;
using ModelContextProtocol.Server;

public static class MyEditorTools
{
    [McpServerTool("EditorMyCustomTool", Description = "私のカスタムエディタツール")]
    public static CallToolResult MyCustomTool(
        [McpArgument(Description = "パラメータ1", Required = true)] string param1,
        [McpArgument(Description = "パラメータ2（オプション）")] int param2 = 0)
    {
        // エディタ操作を実行
        Debug.Log($"カスタムツールを実行：{param1}, {param2}");

        return new CallToolResult
        {
            Content = new List<ContentBlock>
```

```
            {
                new TextContentBlock { Text = $"成功: {param1}" }
            }
        };
    }
}
```

ツールを登録:

```
// GlobalEditorMcpServer 起動後に登録
GlobalEditorMcpServer.Server.RegisterToolsFromClass(typeof(MyEditorTools));
```

3.1.2 パラメータ定義方法

基本型パラメータ

```
[McpServerTool("Example", Description = "例")]
public static CallToolResult Example(
    [McpArgument(Description = "文字列パラメータ", Required = true)] string text,
    [McpArgument(Description = "整数パラメータ")] int number = 0,
    [McpArgument(Description = "浮動小数点パラメータ")] float value = 0f,
    [McpArgument(Description = "ブールパラメータ")] bool flag = false)
{
    // ...
}
```

列挙型パラメータ

```
public enum MyEnum { OptionA, OptionB, OptionC }

[McpServerTool("EnumExample", Description = "列挙型の例")]
public static CallToolResult EnumExample(
    [McpArgument(Description = "列挙オプション")] MyEnum option = MyEnum.OptionA)
{
    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = $"選択: {option}" }
        }
    };
}
```

配列パラメータ

```
[McpServerTool("ArrayExample", Description = "配列の例")]
public static CallToolResult ArrayExample(
    [McpArgument(Description = "文字列配列", Required = true)] string[] items)
{
    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = $"{items.Length} 個のアイテムを受信" }
        }
    };
}
```

Vector3 パラメータ（展開形式）

```
[McpServerTool("VectorExample", Description = "Vector3の例")]
public static CallToolResult VectorExample(
    [McpArgument(Description = "位置")] UnityEngine.Vector3 position)
{
    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = $"位置: {position}" }
        }
    };
}
```

呼び出し時のパラメータ形式:

```
{
  "position_x": 1.0,
  "position_y": 2.0,
  "position_z": 3.0
}
```

3.1.3 カスタム型の定義

カスタム型の定義

```
using Newtonsoft.Json;
using ModelContextProtocol.Server;

public class PlayerData
{
    [JsonProperty("playerName")]
    [McpArgument(Description = "プレイヤー名", Required = true)]
    public string PlayerName;

    [JsonProperty("level")]
```

```
    [McpArgument(Description = "プレイヤーレベル")]
    public int Level;

    [JsonProperty("health")]
    [McpArgument(Description = "ヘルス")]
    public float Health;

    [JsonProperty("position")]
    [McpArgument(Description = "位置")]
    public UnityEngine.Vector3 Position;
}
```

ツールでカスタム型を使用

```
[McpServerTool("SetPlayerData", Description = "プレイヤーデータを設定")]
public static CallToolResult SetPlayerData(
    [McpArgument(Description = "プレイヤーデータ", Required = true)] PlayerData data)
{
    Debug.Log($"プレイヤー: {data.PlayerName}, レベル: {data.Level}");

    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = $"プレイヤー設定完了: {data.PlayerName}" }
        }
    };
}
```

呼び出し形式:

```
{
  "data": {
    "playerName": "Player1",
    "level": 10,
    "health": 100.0,
    "position_x": 0,
    "position_y": 1,
    "position_z": 0
  }
}
```

カスタム型検証ルール

ルール	説明
フィールドには `[JsonProperty]` と `[McpArgument]` の両方が必要	どちらかの属性がないフィールドは無視されます
`[JsonRequired]` または `[McpArgument(Required = true)]` を使用	必須フィールドをマーク
ネストされたカスタム型をサポート	再帰的検証
カスタム型配列をサポート	`PlayerData[]` または `List<PlayerData>`

3.1.4 インスタンスツールの登録

インスタンスツールを使用すると、同じクラスの複数のインスタンスに対して独立したツールセットを登録できます。

インスタンスツールクラスの定義

```
[McpInstanceTool(Name = "Player", Description = "プレイヤーインスタンスツール")]
public class PlayerTools
{
    public string PlayerId { get; set; }
    public int Health { get; set; }
    public int Score { get; set; }

    [McpServerTool(Description = "プレイヤー情報を取得")]
    public CallToolResult GetInfo()
    {
        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = $"プレイヤー {PlayerId}: HP={Health}, Score=
{Score}" }
            }
        };
    }

    [McpServerTool(Description = "ヘルスを設定")]
    public CallToolResult SetHealth(
        [McpArgument(Description = "ヘルス値", Required = true)] int health)
    {
        Health = health;
        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = $"HP を {Health} に設定" }
            }
```

```
        };
    }

    [McpServerTool(Description = "スコアを追加")]
    public void AddScore(
        [McpArgument(Description = "スコア量", Required = true)] int amount)
    {
        Score += amount;
    }
}
```

インスタンスツールの登録

```
// インスタンスを作成
var player1 = new PlayerTools { PlayerId = "P001", Health = 100, Score = 0 };
var player2 = new PlayerTools { PlayerId = "P002", Health = 80, Score = 50 };

// サーバーに登録
GlobalEditorMcpServer.Server.RegisterToolsFromInstance(player1, "player_1");
GlobalEditorMcpServer.Server.RegisterToolsFromInstance(player2, "player_2");

// ツール名形式: {instanceId}.{methodName}
// player_1.GetInfo
// player_1.SetHealth
// player_2.GetInfo
// player_2.SetHealth
```

インスタンスツールの登録解除

```
// 指定インスタンスのすべてのツールを登録解除
GlobalEditorMcpServer.Server.UnregisterInstanceTools("player_1");
```

注意事項

- インスタンス ID は一意である必要があります
- サーバーはインスタンス参照を保持するため、メモリ管理に注意
- 不要になったら登録解除

3.2 ランタイムの拡張

ランタイム拡張はエディタ拡張と同じ方法で行います。

3.2.1 カスタムツールの定義

```
public static class GameTools
{
    [McpServerTool("SpawnEnemy", Description = "敵を生成")]
    public static CallToolResult SpawnEnemy(
```

```
        [McpArgument(Description = "敵のタイプ")] string enemyType,
        [McpArgument(Description = "位置")] UnityEngine.Vector3 position)
    {
        // ゲームランタイムロジック
        var enemy = SpawnManager.SpawnEnemy(enemyType, position);

        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = $"敵を生成: {enemy.name}" }
            }
        };
    }

    [McpServerTool("GetPlayerStatus", Description = "プレイヤーステータスを取得")]
    public static CallToolResult GetPlayerStatus()
    {
        var player = GameManager.Instance.Player;
        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = JObject.FromObject(new
                {
                    health = player.Health,
                    mana = player.Mana,
                    level = player.Level
                }).ToString() }
            }
        };
    }
}
```

登録:

```
_host.Server.RegisterToolsFromClass(typeof(GameTools));
```

3.2.2 インスタンスツールの登録

```
// 定義
[McpInstanceTool(Name = "Unit", Description = "ユニットツール")]
public class UnitTools
{
    public Unit Unit { get; set; }

    [McpServerTool(Description = "ターゲットに移動")]
    public CallToolResult MoveTo(
        [McpArgument(Description = "ターゲット位置")] UnityEngine.Vector3 target)
    {
        Unit.MoveTo(target);
```

```
        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = "移動コマンドを送信" }
            }
        };
    }
}

// 登録
var unit = new UnitTools { Unit = myUnit };
_host.Server.RegisterToolsFromInstance(unit, "unit_001");

// 登録解除
_host.Server.UnregisterInstanceTools("unit_001");
```

四、付録

4.1 McpServerHostOptions 設定表

プロパティ	型	デフォルト値	説明
`Port`	int	3000	HTTP サービスポート
`ServerName`	string	"UnityMCP"	MCP サーバー名
`ServerVersion`	string	"1.0.0"	サーバーバージョン
`Instructions`	string	-	AI に送信する指示
`LogLevel`	LogLevel	Information	ログ出力レベル

4.2 イベント一覧

イベント	シグネチャ	発生タイミング
`OnServerStarted`	`Action`	サーバー起動成功時
`OnServerStopped`	`Action`	サーバー停止時
`OnServerError`	`Action<string>`	サーバーエラー発生時

4.3 戻り値の型

成功結果

```
return new CallToolResult
{
    Content = new List<ContentBlock>
    {
        new TextContentBlock { Text = "操作成功" }
    }
};
```

エラー結果

```
return new CallToolResult
{
    IsError = true,
    Content = new List<ContentBlock>
    {
        new TextContentBlock { Text = "エラーメッセージ" }
    }
};
```

画像を含む結果

```
return new CallToolResult
{
    Content = new List<ContentBlock>
    {
        new ImageContentBlock
        {
            Data = base64ImageData,
            MimeType = "image/png"
        }
    }
};
```

4.4 注意事項

1. **スレッドセーフ**: MCP ツールはバックグラウンドスレッドで実行されます。Unity オブジェクトを操作する場合は `MainThreadDispatcher` を使用するか、メインスレッドでの実行を確認してください
2. **ライフサイクル**: `McpServerHost` は自動的に `Application.quitting` をリッスンし、アプリケーション終了時にクリーンアップします
3. **セキュリティ**: デフォルトでは `localhost` でリッスンし、外部接続を受け付けません。本番環境では慎重に設定してください
4. **パフォーマンス**: 大量のツール呼び出しはパフォーマンスに影響する可能性があるため、呼び出し頻度を適切に管理してください
5. **エラー処理**: ツール内部の例外はキャッチされ、エラー結果として返されます。サーバーはクラッシュしません

4.5 ファイル構造

```
Assets/McpForUnity/
├── Core/
│   ├── IMcpServerHost.cs          # サーバーインターフェース
│   ├── McpServerHost.cs           # サーバー実装
│   ├── McpServerHostOptions.cs    # 設定クラス
│   ├── McpException.cs            # 例外型
│   ├── Throw.cs                   # パラメータ検証
│   └── Protocol/                  # MCP プロトコル型
├── Server/
│   ├── McpServer.cs               # コアサーバー
│   ├── McpServerOptions.cs        # サーバー設定
│   └── Tools/
│       └── Attributes.cs          # ツール属性定義
├── Transport/
│   ├── ITransport.cs              # 転送インターフェース
│   └── HttpListenerServerTransport.cs
├── Editor/
│   ├── Tools/
│   │   └── EditorToolsList.cs     # 80 エディタツール
│   ├── GlobalEditorMcpServer.cs   # エディタサーバー
│   └── Window/
│       └── McpServerEditorWindow.cs
├── Utilities/
│   ├── UnityLogger.cs             # ログ実装
│   └── MainThreadDispatcher.cs    # メインスレッドディスパッチ
└── Samples/
    └── MCPExampleUsage.cs         # 使用例
```

4.6 よくある質問

Q: ポートが使用中の場合はどうすればいいですか?

A: `Port` 設定を別のポートに変更するか、使用中のポートを解放してください。

Q: Newtonsoft.Json が見つかりません

A: `com.unity.nuget.newtonsoft-json` パッケージがインストールされていることを確認:

```
Window > Package Manager > + > Add package by name
入力: com.unity.nuget.newtonsoft-json
```

Q: ツールで MonoBehaviour にアクセスするにはどうすればいいですか?

A: `GameObject.Find()` またはシングルトンパターンを使用してシーン内のオブジェクト参照を取得してください。

Q: ツールパラメータで複雑なオブジェクトを渡すにはどうすればいいですか?

A: カスタム型を定義し、`[JsonProperty]` と `[McpArgument]` でフィールドをアノテーションしてください。

五、AIアシスト開発

MCP For Unity は AI クライアントを通じたアシスト開発をサポートしています。プロジェクト設定ファイルを作業ディレクトリにコピーするだけで、AI アシスタントがプロジェクト構造を理解し、開発を支援できます。

5.1 クイックスタート

ステップ1：設定ファイルのコピー

`AGENTS.md` をプロジェクトルートにコピー：

```
# macOS / Linux
cp Assets/McpForUnity/AGENTS.md ./

# Windows
copy Assets\McpForUnity\AGENTS.md .
```

ステップ2：AI クライアントの起動

MCP プロトコルをサポートする AI クライアント（OpenCode、Claude Desktop など）でプロジェクトディレクトリを開きます。

ステップ3：開発開始

AI アシスタントは自動的に `AGENTS.md` からプロジェクト情報を読み取ります：

- コード規約と命名規則
- プロジェクト構造とファイル場所
- ビルドとテストコマンド
- API 使用方法

5.2 推奨ツール

ツール	説明
OpenCode	推奨、ネイティブ MCP プロトコルサポート
Claude Desktop	MCP サーバー設定が必要
Cursor	MCP 拡張をサポート
VS Code + MCP プラグイン	MCP 拡張のインストールが必要

5.3 開発ワークフロー

```
1. AI が AGENTS.md を読み取る → プロジェクト規約を理解
2. MCP Server を起動 → AI が Unity ツールを呼び出し可能に
3. 要件を説明 → AI がコードを生成/修正
4. AI がツールを呼び出す → コードを検証/テスト
5. 反復 → 開発完了
```

5.4 サンプルシナリオ

シナリオ	例のプロンプト
新規ツール追加	"シーン階層を JSON でエクスポートするエディタツールを追加"
問題修正	"EditorInstantiatePrefab ツールがエラーを投げています、修正して"
コードリファクタリング	"このツールクラスをプロジェクト規約に従ってリファクタリング"
テスト作成	"EditorCreateGameObject の単体テストを作成"
ドキュメント追加	"新しいツールにコメントと使用ドキュメントを追加"

5.5 注意事項

- AI アシスタントは MCP プロトコルを通じて Unity エディタを直接操作できます
- AI がコードを修正した後はコンパイルチェックを実行することを推奨
- 複雑な操作は段階的に進め、検証とロールバックを容易に

ドキュメントバージョン: 1.0.3
最終更新: 2026-03-02